PROBLEM STATEMENT:

In today's fast-paced financial markets, investors and analysts require efficient and user-friendly tools to perform comprehensive stock market analysis and make informed investment decisions. Traditional methods often involve complex software and significant manual effort, lacking the real-time interactivity and integration capabilities necessary for dynamic market conditions. There is a need for an intuitive, accessible platform that can seamlessly integrate with financial data sources, perform advanced data analytics, and provide interactive visualizations and predictive insights. This project aims to develop a web-based stock market analytics application using Streamlit and Python, which addresses these needs by offering a comprehensive, real-time, and user-friendly solution for stock market analysis.

ABSTRACT:

This paper presents the development of an interactive web application for comprehensive stock market analytics, leveraging Streamlit, a prominent Python library. The application integrates financial data retrieval, technical analysis, and machine learning to offer insightful visualizations and predictive analytics for stock market data. Historical stock data is fetched using the Yahoo Finance API, allowing users to input stock symbols and date ranges. Key technical indicators such as Volume Weighted Average Price (VWAP), Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI), Bollinger Bands, On-Balance Volume (OBV), Simple Moving Average (SMA), Exponential Moving Average (EMA), Average True Range (ATR), Stochastic Oscillator, and Chaikin Money Flow (CMF) are calculated and visualized.

The application also facilitates comparative analysis of multiple stocks and includes advanced machine learning capabilities. A Random Forest Regressor model is employed to predict future stock closing prices based on selected features, with the model's performance evaluated using Mean Squared Error (MSE) and R-squared (R^2) metrics. Data preprocessing functionalities such as binning, normalization, and both random and stratified sampling are implemented to enhance the analysis process.

Dynamic and animated charts are created using Plotly, enhancing user engagement and comprehension of complex financial data. The application aims to serve both novice and experienced traders by providing a robust tool for analyzing stock performance, predicting prices, and making informed trading decisions. The user-friendly interface and interactive visualizations make this application a valuable asset for financial analysis.

STREAMLIT:

The purpose of Streamlit in this project is to provide a straightforward and efficient platform for developing an interactive web application that allows users to perform comprehensive stock market analysis. It facilitates the creation of real-time, dynamic visualizations and user interfaces without requiring extensive front-end development knowledge. Streamlit integrates seamlessly with Python libraries, enabling rapid prototyping, easy deployment, and immediate user feedback, which enhances the overall user experience and accessibility of the stock market analytics tool.

PYTHON:

Python is preferred for this project due to its simplicity, readability, and extensive ecosystem of libraries that cater specifically to data analysis, machine learning, and web development. Libraries such as Pandas, NumPy, Scikit-learn, and Plotly provide robust tools for data manipulation, statistical analysis, model building, and interactive visualization. Additionally, Python's integration with Streamlit enables the rapid development of user-friendly web applications. Its widespread use in the finance and data science communities ensures ample resources, community support, and continuous improvements, making it an ideal choice for building a comprehensive stock market analytics platform.

OBJECTIVES:

1. Data Retrieval:

- Implement a system to fetch historical stock market data for specified symbols and date ranges using the Yahoo Finance API.
- Ensure the data retrieval process is efficient, reliable, and can handle potential errors or data unavailability gracefully.

2. Technical Analysis:

- Calculate and visualize key technical indicators such as Volume Weighted Average Price (VWAP), Moving Average Convergence Divergence (MACD), Relative Strength Index (RSI), Bollinger Bands, On-Balance Volume (OBV), Simple Moving Average (SMA), Exponential Moving Average (EMA), Average True Range (ATR), Stochastic Oscillator, and Chaikin Money Flow (CMF).
- Provide clear and informative visualizations for these indicators to assist users in understanding stock performance and trends.

3. Comparative Analysis:

- Enable users to compare the closing prices of multiple stocks over a specified date range.
- Provide visual tools to highlight differences and similarities in stock performance.

4. Predictive Analytics:

- Develop a machine learning model, specifically a Random Forest Regressor, to predict future stock closing prices based on user-selected features.
- Evaluate the model's performance using appropriate metrics such as Mean Squared Error (MSE) and R-squared (R^2) scores.
- Create an interface for users to input feature values and obtain predicted stock prices.

5. Data Preprocessing:

- Implement binning techniques to categorize and analyze the distribution of selected features.
- Provide options for data normalization to ensure consistent scaling of features.
- Incorporate both random and stratified sampling techniques to generate representative data subsets for analysis and model training.

6. Interactive Visualizations:

- Utilize Plotly to create dynamic and interactive charts for various time series data and technical indicators.
- Develop animations for time series plots to enhance user engagement and facilitate a better understanding of data trends over time.

7. User Experience:

- Design a user-friendly interface using Streamlit that simplifies the interaction with complex financial data and analytical tools.
- Ensure the application is accessible to both novice and experienced traders, providing intuitive controls and informative outputs.

8. Robustness and Reliability:

- Ensure the application handles edge cases and potential errors gracefully, providing meaningful feedback to users in case of issues.
- Optimize the performance and responsiveness of the application to handle large datasets and complex computations efficient.

OVERVIEW OF THE DATASET:

This dataset encompasses the price history and trading volumes of fifty stocks listed in the NIFTY 50 index from the National Stock Exchange (NSE) of India. Spanning from January 1, 2000, to April 30, 2021, it provides a detailed daily record of pricing and trading values for each stock. The dataset covers a diverse range of companies, including ADANIPORTS, ASIANPAINT, AXISBANK, BAJAJ-AUTO, BAJAJFINSV, BAJFINANCE, BHARTIARTL, BPCL, BRITANNIA, CIPLA, COALINDIA, DRREDDY, EICHERMOT, GAIL, GRASIM, HCLTECH, HDFC, HDFCBANK, HEROMOTOCO, HINDALCO, HINDUNILVR, ICICIBANK, INDUSINDBK, INFRATEL, INFY, IOC, ITC, JSWSTEEL, KOTAKBANK, LT, MARUTI, MM, NESTLEIND, NTPC, ONGC, POWERGRID, RELIANCE, SBIN, SHREECEM, SUNPHARMA, TATAMOTORS, TATASTEEL, TCS, TECHM, TITAN, ULTRACEMCO, UPL, VEDL, WIPRO, and ZEEL.

1. Date: The date of the trading day, providing a chronological timeline of stock market activity.

2. Symbol: The unique symbol or ticker assigned to each stock, facilitating identification and trading on the exchange.

3. Series: The categorization of the stock based on its type, such as equity shares or preference shares, which may impact trading regulations and investor rights.

4. Prev Close: The closing price of the stock on the previous trading day, serving as a reference point for analyzing price movements.

5. Open: The opening price of the stock at the beginning of the trading day, indicating the initial valuation at which trading commenced.

6. High: The highest price reached by the stock during the trading day, reflecting the peak value achieved by the stock.

7. Low: The lowest price reached by the stock during the trading day, representing the minimum value recorded for the stock.

8. Last: The last traded price of the stock during the trading day, providing the most recent valuation before the market closes.

9. Close: The closing price of the stock on the trading day, indicating the final valuation at the end of the trading session.

10. VWAP: Volume Weighted Average Price, calculated as the average price of the stock weighted by trading volume, providing insight into the average transaction price.

11. Volume: The total number of shares traded for the stock on the trading day, reflecting the level of market activity and liquidity.

12. Turnover: The total value of stocks traded for the stock on the trading day, representing the overall monetary value exchanged.

13. Trades: The total number of trades executed for the stock on the trading day, indicating the level of trading activity and investor participation.

14. Deliverable Volume: The volume of shares that are delivered (settled) on the trading day, providing insight into the actual transactions executed.

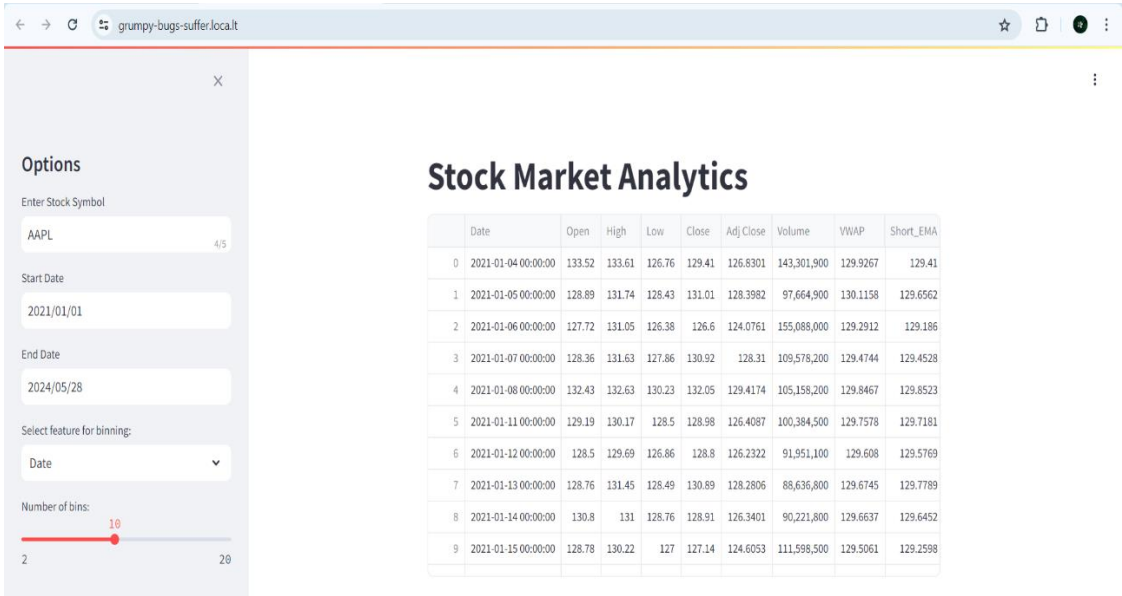
15. %Deliverable: The percentage of deliverable volume to total volume traded, indicating the proportion of shares that result in actual delivery.

16. Company Name: The name of the company associated with the stock, providing context and identification for the listed entity.

17. Industry: The industry or sector to which the company belongs, offering insight into the business operations and market positioning.

18. ISIN Code: The International Securities Identification Number assigned to the stock, providing a unique identifier for securities traded globally.

DASHBOARD:



	Date	Open	High	Low	Close	Adj Close	Volume	VWAP	Short_EMA
0	2021-01-04 00:00:00	133.52	133.61	126.76	129.41	126.8301	143,301,900	129.9267	129.41
1	2021-01-05 00:00:00	128.89	131.74	128.43	131.01	128.3982	97,664,900	130.1158	129.6562
2	2021-01-06 00:00:00	127.72	131.05	126.38	126.6	124.0761	155,088,000	129.2912	129.186
3	2021-01-07 00:00:00	128.36	131.63	127.86	130.92	128.31	109,578,200	129.4744	129.4528
4	2021-01-08 00:00:00	132.43	132.63	130.23	132.05	129.4174	105,158,200	129.8467	129.8523
5	2021-01-11 00:00:00	129.19	130.17	128.5	128.98	126.4087	100,384,500	129.7578	129.7181
6	2021-01-12 00:00:00	128.5	129.69	126.86	128.8	126.2322	91,951,100	129.608	129.5769
7	2021-01-13 00:00:00	128.76	131.45	128.49	130.89	128.2806	88,636,800	129.6745	129.7789
8	2021-01-14 00:00:00	130.8	131	128.76	128.91	126.3401	90,221,800	129.6637	129.6452
9	2021-01-15 00:00:00	128.78	130.22	127	127.14	124.6053	111,598,500	129.5061	129.2598

The dashboard presents a comprehensive analysis of stock market data, offering users interactive features to explore the price history and trading volumes of various stocks listed in the NIFTY 50 index from the National Stock Exchange (NSE) of India. Users can input parameters like stock symbol, date range, and select analysis options such as feature selection, binning, normalization, and sampling techniques. Visualizations include dynamic charts illustrating closing prices, volume traded, MACD indicator, RSI, Bollinger Bands, and more, providing insights into stock performance over time. Additionally, users can utilize machine learning models to predict stock prices and evaluate model accuracy. Animated time series plots enable dynamic observation of trends, enhancing the dashboard's usability for data analysis and decision-making.

CLOSING PRICE OVER TIME:

CODE:

```
st.subheader("Closing Price Over Time")

fig_close = go.Figure(data=go.Scatter(x=data['Date'],
y=data['Close'], mode='lines'))

fig_close.update_layout(title=f"Closing Price of {symbol}")

st.plotly_chart(fig_close, use_container_width=True)
```

DESCRIPTION:

It's a subplot in the Streamlit dashboard titled "Closing Price Over Time". It utilizes Plotly to generate a line plot (`go.Scatter`) representing the closing prices of a selected stock (`data['Close']`) over time (`data['Date']`). The title of the plot is dynamically set to "Closing Price of [selected stock symbol]". Finally, the plot is rendered using `st.plotly_chart`, ensuring it fits within the width of the container.

INFERENCE:

The inference from the output of this code segment would be the trend of the closing prices of the selected stock over time. By visualizing the closing prices as a line plot, we can observe whether the stock's prices have been increasing, decreasing, or remaining relatively stable over the specified time period. This visualization helps in understanding the historical performance of the stock and identifying any patterns or trends that may exist.

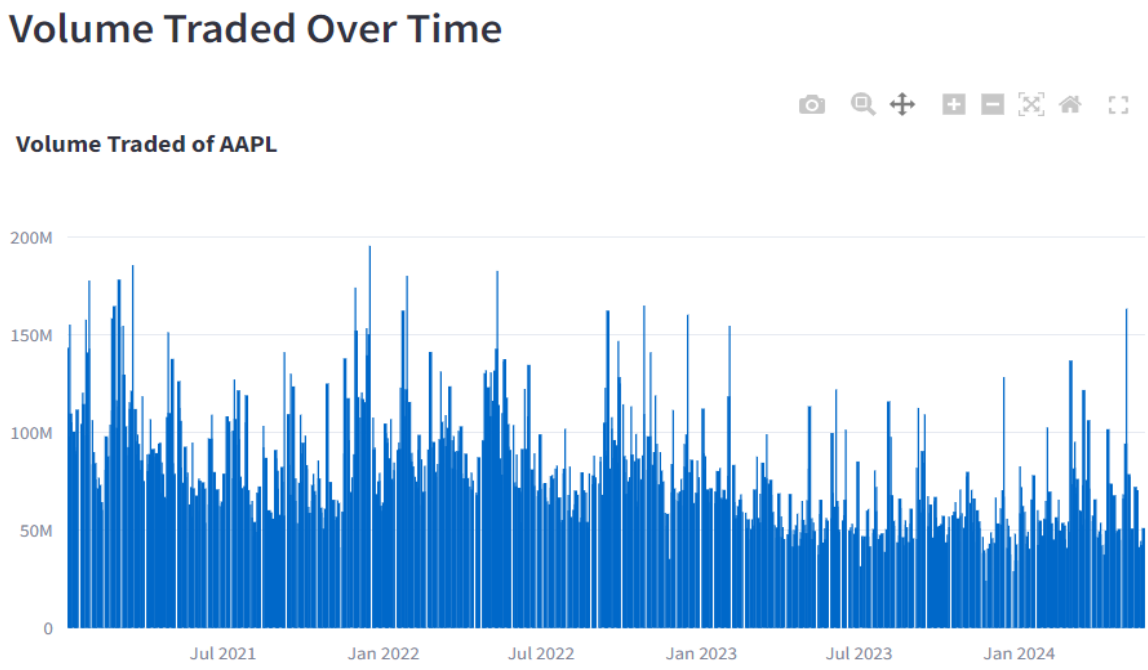
VOLUME TRADED OVER TIME:

CODE:

```
st.subheader("Volume Traded Over Time")

fig_volume = go.Figure(data=go.Bar(x=data['Date'],
y=data['Volume']))

fig_volume.update_layout(title=f"Volume Traded of {symbol}")

st.plotly_chart(fig_volume, use_container_width=True)
```

DESCRIPTION:

This plot focuses on visualizing the volume traded over time for a selected stock. It creates a subplot titled "Volume Traded Over Time" and uses Plotly to generate a bar chart (go.Bar) with the x-axis representing dates (data['Date']) and the y-axis representing the volume traded (data['Volume']). The title of the plot dynamically includes the selected stock symbol. Finally, the plot is rendered using st.plotly_chart, ensuring it fits within the width of the container.



INFERENCE:

The inference from this code segment is the visualization of the volume traded over time for the selected stock symbol. The bar chart shows the volume of trades on the y-axis and the corresponding dates on the x-axis. By observing this chart, one can analyze the trading activity of the stock over the specified time period. Increases or decreases in trading volume can indicate shifts in market sentiment or increased investor interest, providing valuable insights into the stock's liquidity and trading patterns.

OPENING PRIE VS CLOSING PRICE OVER TIME:

CODE:

```
st.subheader("Opening vs Closing Prices Over Time")

fig_open_close = go.Figure()

fig_open_close.add_trace(go.Scatter(x=data['Date'],
y=data['Open'], mode='lines', name='Open'))

fig_open_close.add_trace(go.Scatter(x=data['Date'],
y=data['Close'], mode='lines', name='Close'))

    fig_open_close.update_layout(title='Opening vs Closing Prices
Over Time')

    st.plotly_chart(fig_open_close, use_container_width=True)
```

DESCRIPTION:

It's a subplot titled "Opening vs Closing Prices Over Time" in the Streamlit dashboard. It initializes a Plotly figure (`fig_open_close`) and adds two line plots using `go.Scatter`. The first plot represents the opening prices (`data['Open']`) over time (`data['Date']`), labeled as 'Open'. The second plot represents the closing prices (`data['Close']`) over the same time period, labeled as 'Close'. The subplot is then given a title, "Opening vs Closing Prices Over Time", and rendered using `st.plotly_chart`, ensuring it fits within the width of the container.

INFERENCE:

This code segment is the comparison between the opening and closing prices of the selected stock symbol over time. The line chart displays the trend of both opening and closing prices on the y-axis against the corresponding dates on the x-axis. By analyzing this chart, one can gain insights into the price movements of the stock throughout the specified time period. Discrepancies or similarities between opening and closing prices can indicate intraday volatility, market sentiment, and potential trading opportunities.

OHLC CHART:

CODE:

```
st.subheader("OHLC Chart")
fig_candlestick = go.Figure(data=[go.Candlestick(x=data['Date'],
open=data['Open'],
high=data['High'],
low=data['Low'],
close=data['Close'])])
fig_candlestick.update_layout(title='OHLC Chart')
st.plotly_chart(fig_candlestick, use_container_width=True)
```

DESCRIPTION:

The correct term is "OHLC" chart, which stands for Open-High-Low-Close chart. It is a type of financial chart used to illustrate the movements of a security's price over a specific time period. Each candlestick in an OHLC chart represents the open, high, low, and close prices of a security for a particular period, typically a day. The body of the candlestick shows the opening and closing prices, while the upper and lower "wicks" or "shadows" represent the highest and lowest prices reached during the period, respectively. These charts are commonly used in technical analysis to analyze price trends and make trading decisions.

INFERENCE:

It generates an OHLC (Open-High-Low-Close) chart, is a visualization of the price movement of the selected stock symbol over time. The OHLC chart displays the opening, high, low, and closing prices of the stock for each trading day. By examining the OHLC chart, one can observe the price range (from the lowest to the highest), as well as the opening and closing prices, providing insights into price volatility, market sentiment, and potential trading patterns such as trends, reversals, or consolidation periods.

VWAP:

CODE:

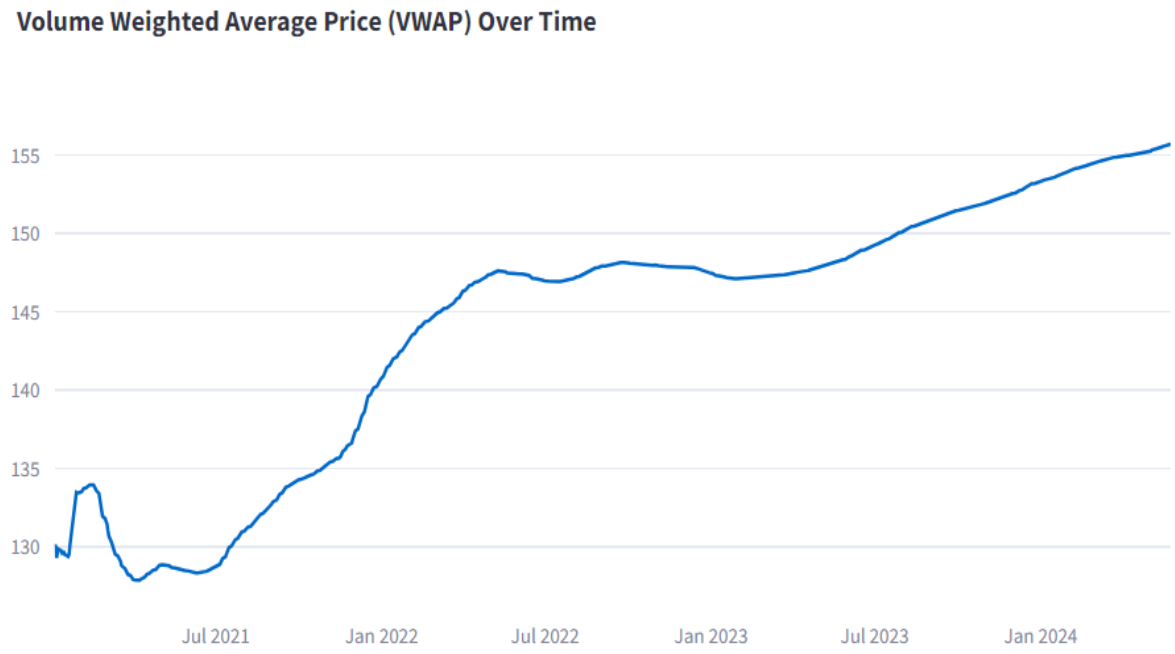
```
    st.subheader("Volume Weighted Average Price (VWAP) Over Time")

    fig_vwap = go.Figure()

    fig_vwap.add_trace(go.Scatter(x=data['Date'], y=data['VWAP'],
mode='lines',        name='VWAP'))

    fig_vwap.update_layout(title='Volume Weighted Average Price
(VWAP) Over Time')

    st.plotly_chart(fig_vwap, use_container_width=True)
```

DESCRIPTION:

This code segment creates a plot of the Volume Weighted Average Price (VWAP) over time using Plotly. It begins by initializing a Plotly figure object (`fig_vwap`). Next, a scatter plot is added to the figure using the `go.Scatter` function, where the x-axis represents the dates (`data['Date']`) and the y-axis represents the VWAP (`data['VWAP']`). The mode is set to 'lines' to connect the data points with lines.

The layout of the figure is updated to set the title to "Volume Weighted Average Price (VWAP) Over Time" using `fig_vwap.update_layout`. Finally, the plot is rendered in the Streamlit app using `st.plotly_chart(fig_vwap, use_container_width=True)`, which displays the plot with the specified layout settings within the Streamlit interface, utilizing the available container width.

Volume Weighted Average Price (VWAP) Over Time



INFERENCE:

The code segment generates a plot showing the Volume Weighted Average Price (VWAP) over time. VWAP is a trading benchmark used by traders to assess the average price at which a security has traded throughout the day, based on both volume and price. This visualization provides insights into the average price levels at which significant trading activity has occurred over the specified time period. Analyzing the VWAP over time can help traders understand price trends, support and resistance levels, as well as potential buying or selling opportunities based on deviations from the VWAP.

TRADE ANALYSIS:

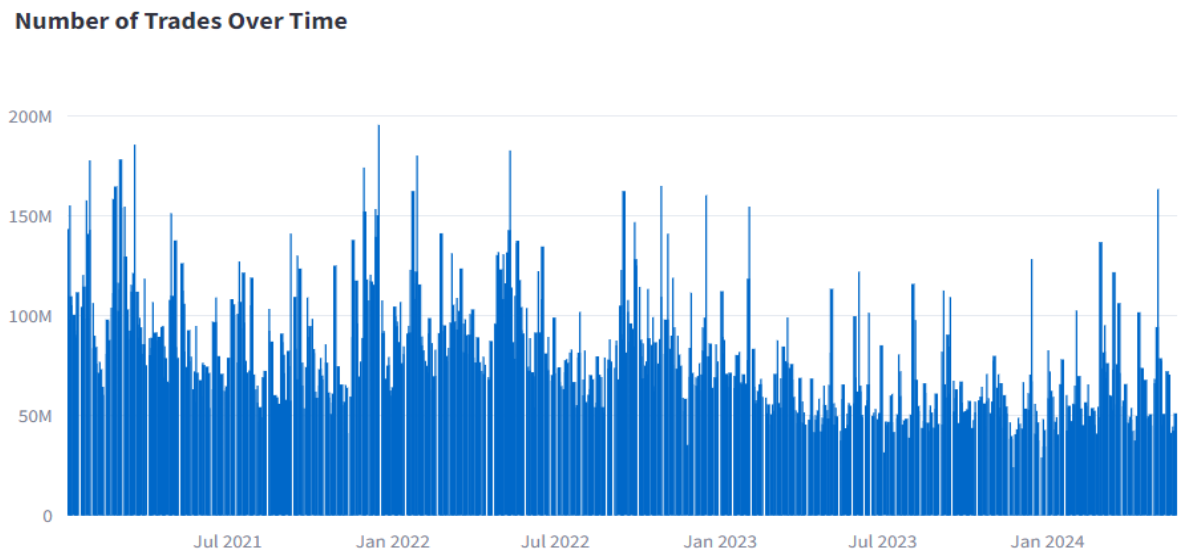
CODE:

```
st.subheader("Number of Trades Over Time")

fig_trades = go.Figure()

fig_trades.add_trace(go.Bar(x=data['Date'],    y=data['Volume'],
name='Trades'))

fig_trades.update_layout(title='Number of Trades Over Time')

st.plotly_chart(fig_trades, use_container_width=True)
```

DESCRIPTION:

It begins by creating a Plotly figure object (fig_trades). Then, a bar plot is added to the figure using the go.Bar function, where the x-axis represents the dates (data['Date']) and the y-axis represents the number of trades (data['Volume']). The name of the bar is set to 'Trades'. The layout of the figure is updated to set the title to "Number of Trades Over Time" using fig_trades.update_layout. Finally, the plot is rendered in the Streamlit app using st.plotly_chart(fig_trades, use_container_width=True), which displays the bar chart within the Streamlit interface, utilizing the available container width.

Number of Trades Over Time



INFERENCE:

Analyzing trades involves calculating metrics like daily returns, which show the percentage change in the stock price from one day to the next. This can help in understanding the stock's volatility and in developing trading strategies.

COMPARATIVE ANALYSIS:

CODE:

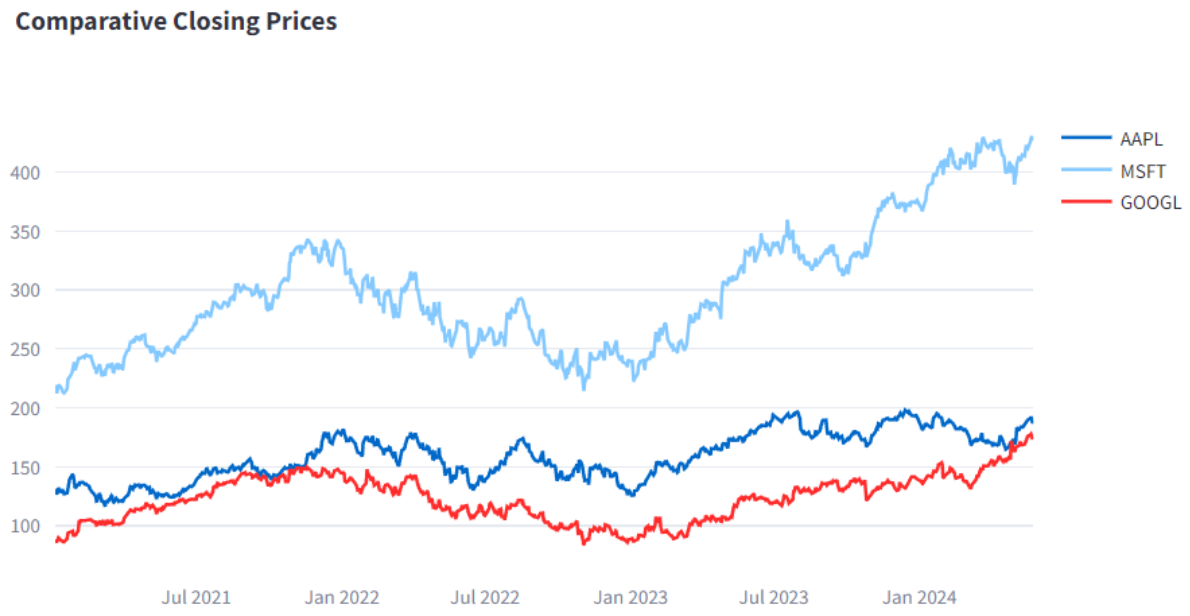
```
    symbols_list = ['AAPL', 'MSFT', 'GOOGL']  # Add more symbols as needed

    st.subheader("Comparative Closing Prices")

    fig_compare = go.Figure()

    for symbol in symbols_list:

       compare_data = load_data(symbol, start_date, end_date)

       fig_compare.add_trace(go.Scatter(x=compare_data['Date'], y=compare_data['Close'], mode='lines', name=symbol))

       fig_compare.update_layout(title='Comparative Closing Prices')

       st.plotly_chart(fig_compare, use_container_width=True)
```

DESCRIPTION:

It creates a Plotly figure object (fig_compare) to visualize the comparative closing prices. Within a loop iterating over each symbol in the symbols_list, it loads the data for each symbol using a function load_data() with specified start and end dates. For each symbol, a line plot is added to the figure, where the x-axis represents the dates (compare_data['Date']) and the y-axis represents the closing prices (compare_data['Close']). The name of each line plot is set to the corresponding symbol.

After adding all the traces, the layout of the figure is updated to set the title to "Comparative Closing Prices". Finally, the plot is rendered in the Streamlit app using st.plotly_chart(fig_compare, use_container_width=True), which displays the comparative line chart within the Streamlit interface, utilizing the available container width.

Comparative Closing Prices



INFERENCE:

Comparing the stock's performance against a benchmark, such as the S&P 500, helps in evaluating how well the stock is doing relative to the broader market. This can provide context to the stock's movements and aid in portfolio performance assessment.

MACD INDICATORS:

CODE:

```
     def calculate_macd(data, short_window=12, long_window=26,
signal_window=9):

        data['Short_EMA'] = data['Close'].ewm(span=short_window,
adjust=False).mean()

        data['Long_EMA'] = data['Close'].ewm(span=long_window,
adjust=False).mean()

        data['MACD'] = data['Short_EMA'] - data['Long_EMA']

        data['Signal_Line'] = data['MACD'].ewm(span=signal_window,
adjust=False).mean()

        return data


    data = calculate_macd(data)


    st.subheader("MACD Indicator")

    fig_macd = go.Figure()

    fig_macd.add_trace(go.Scatter(x=data['Date'], y=data['MACD'],
mode='lines', name='MACD'))

    fig_macd.add_trace(go.Scatter(x=data['Date'],
y=data['Signal_Line'], mode='lines', name='Signal Line'))

    fig_macd.update_layout(title='MACD Indicator')

    st.plotly_chart(fig_macd, use_container_width=True)
```

DESCRIPTION:

The calculate_macd function takes the stock data as input along with optional parameters for the short window, long window, and signal window (default values are 12, 26, and 9, respectively). Within the function, it computes the Short EMA (Exponential Moving Average) and Long EMA for the closing prices based on the specified window sizes. Then, it calculates the MACD line by taking the difference between the Short EMA and Long EMA. Finally, it computes the Signal Line by applying an EMA to the MACD line.

After calculating the MACD indicator for the data, it plots the MACD line and Signal Line over time using a Plotly figure (fig_macd). The x-axis represents the dates (data['Date']), and the y-axis represents the MACD values (data['MACD'] and data['Signal_Line']).

MACD Indicator

INFERENCE:

MACD Line: The difference between the 12-day EMA and the 26-day EMA.

Signal Line: A 9-day EMA of the MACD Line.

A bullish signal occurs when the MACD crosses above the Signal Line, and a bearish signal occurs when it crosses below.

RSI CHARTS:

CODE:

```
def calculate_rsi(data, window=14):
    delta = data['Close'].diff(1)
    gain = (delta.where(delta > 0, 0)).fillna(0)
    loss = (-delta.where(delta < 0, 0)).fillna(0)
    avg_gain = gain.rolling(window=window).mean()
    avg_loss = loss.rolling(window=window).mean()
    rs = avg_gain / avg_loss
    data['RSI'] = 100 - (100 / (1 + rs))
    return data

data = calculate_rsi(data)

st.subheader("Relative Strength Index (RSI)")
fig_rsi = go.Figure()
fig_rsi.add_trace(go.Scatter(x=data['Date'],      y=data['RSI'],
mode='lines', name='RSI'))
fig_rsi.update_layout(title='Relative  Strength  Index  (RSI)',
yaxis=dict(range=[0, 100]))
st.plotly_chart(fig_rsi, use_container_width=True)
```

DESCRIPTION:

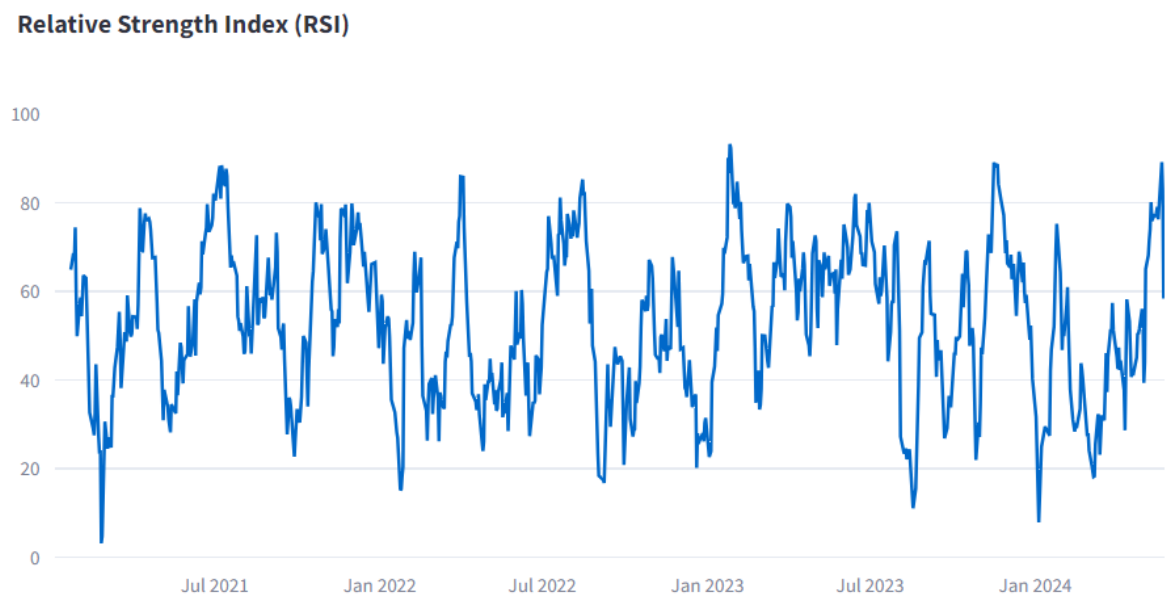
This code segment calculates the Relative Strength Index (RSI) for the given stock data. The calculate_rsi function takes the stock data as input along with an optional parameter for the window size (default value is 14).

Within the function, it calculates the price differences (delta) between consecutive closing prices. It separates the gains and losses from the price differences and computes the average gain (avg_gain) and average loss (avg_loss) over the specified window size using the rolling mean. Then, it calculates the Relative Strength (RS) as the ratio of average gain to average loss.

formula:

$$RSI = 100 - (100 / (1 + RS)).$$

Relative Strength Index (RSI)



INFERENCE:

Overbought: RSI above 70 suggests the stock may be overbought.

Oversold: RSI below 30 suggests the stock may be oversold.

BOLLINGER BANDS:

CODE:

```
    def calculate_bollinger_bands(data, window=20):

        data['MA20'] = data['Close'].rolling(window=window).mean()

        data['STD20'] = data['Close'].rolling(window=window).std()

        data['Upper_Band'] = data['MA20'] + (data['STD20'] * 2)

        data['Lower_Band'] = data['MA20'] - (data['STD20'] * 2)

        return data


    data = calculate_bollinger_bands(data)


    st.subheader("Bollinger Bands")

    fig_bollinger = go.Figure()

    fig_bollinger.add_trace(go.Scatter(x=data['Date'],
y=data['Close'], mode='lines', name='Close'))

    fig_bollinger.add_trace(go.Scatter(x=data['Date'],
y=data['Upper_Band'], mode='lines', name='Upper Band'))

    fig_bollinger.add_trace(go.Scatter(x=data['Date'],
y=data['Lower_Band'], mode='lines', name='Lower Band'))

    fig_bollinger.update_layout(title='Bollinger Bands')

    st.plotly_chart(fig_bollinger, use_container_width=True)
```

DESCRIPTION:

This code segment calculates the Bollinger Bands for the given stock data. The calculate_bollinger_bands function takes the stock data as input along with an optional parameter for the window size (default value is 20).

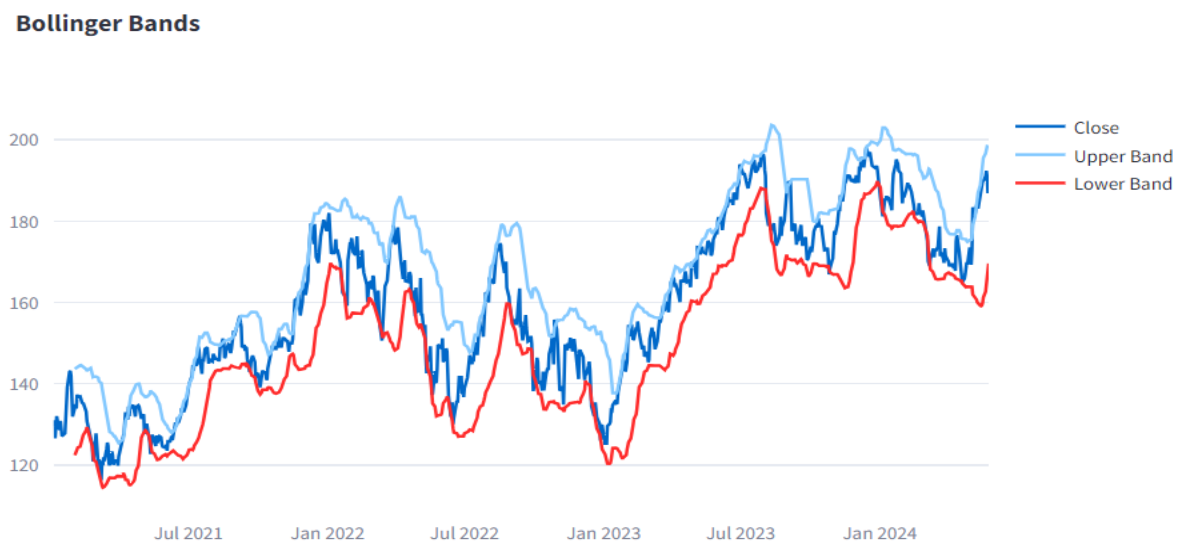
Within the function, it calculates the rolling mean (MA20) and rolling standard deviation (STD20) of the closing prices over the specified window size using the rolling method. Then, it computes the Upper and Lower Bollinger Bands using the formulas:

$$\text{Upper Band} = \text{MA20} + (2 * \text{STD20})$$

$$\text{Lower Band} = \text{MA20} - (2 * \text{STD20})$$

After calculating the Bollinger Bands for the data, it plots the closing prices, Upper Band, and Lower Band over time using a Plotly figure (fig_bollinger). The x-axis represents the dates (data['Date']), and the y-axis represents the respective values. The closing prices are plotted as a line (mode='lines'), and the Upper and Lower Bands are also plotted as lines.

Bollinger Bands



INFERENCE:

Price touching the upper band: Indicates the stock may be overbought.

Price touching the lower band: Indicates the stock may be oversold.

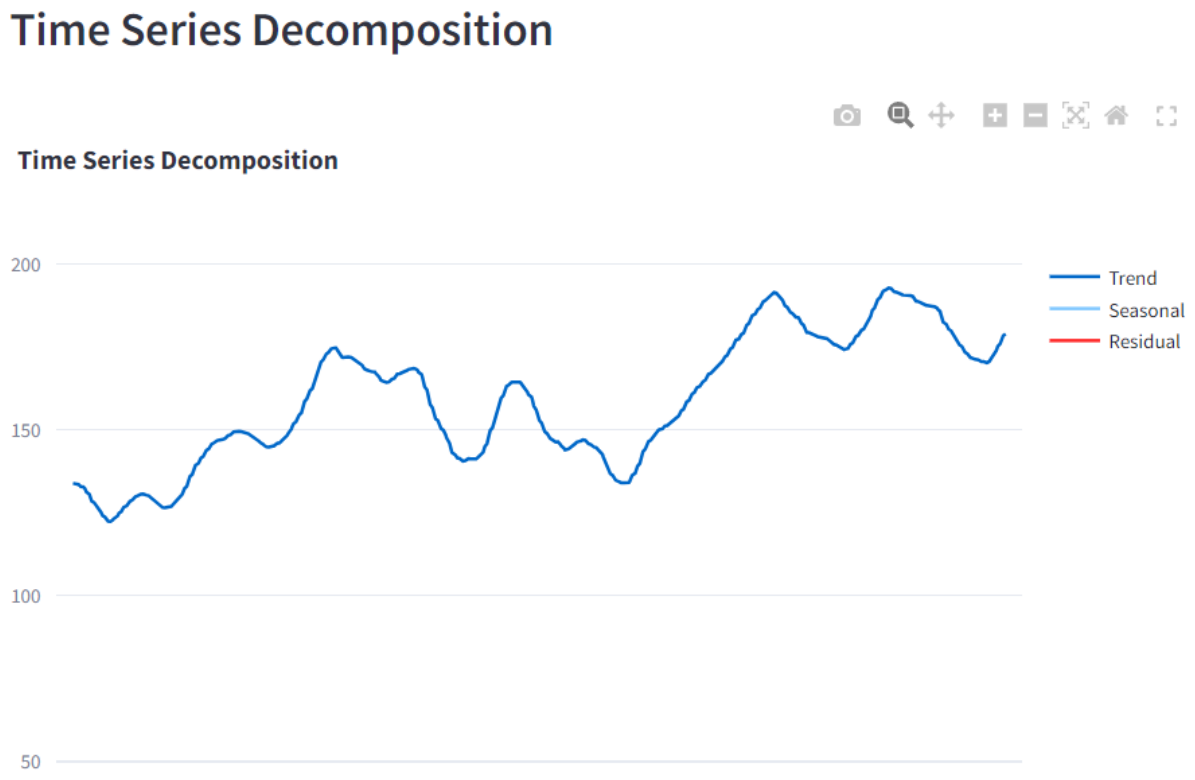
TIME-SERIES DECOMPOSITION:

CODE:

```
  st.subheader("Time Series Decomposition")

    decomposition    =    seasonal_decompose(data['Close'].dropna(),
model='multiplicative', period=30)

    fig_decomp = go.Figure()


    fig_decomp.add_trace(go.Scatter(x=data['Date'],
y=decomposition.trend, mode='lines', name='Trend'))

    fig_decomp.add_trace(go.Scatter(x=data['Date'],
y=decomposition.seasonal, mode='lines', name='Seasonal'))

    fig_decomp.add_trace(go.Scatter(x=data['Date'],
y=decomposition.resid, mode='lines', name='Residual'))


    fig_decomp.update_layout(title='Time    Series    Decomposition',
height=600)

    st.plotly_chart(fig_decomp, use_container_width=True)
```

DESCRIPTION:

The code snippet performs time series decomposition on the 'Close' price data within a Streamlit app. It uses a multiplicative model to break down the series into trend, seasonal, and residual components, assuming a periodicity of 30 days. The results are plotted using Plotly, displaying each component as a separate line trace in a figure titled "Time Series Decomposition." The figure is adjusted to fit the container width for clear visualization.



INFERENCE:

The time series decomposition of the 'Close' price data reveals distinct components: the trend shows a general upward movement with some fluctuations, the seasonal component captures regular patterns or cycles, and the residual component reflects random noise and irregular variations. This breakdown helps identify underlying patterns in the data, separating long-term trends from seasonal effects and noise, thereby providing insights into the price movements.

ON-BALANCE -VOLUME:

CODE:

```
 def calculate_obv(data):
        data['OBV']      =      (np.sign(data['Close'].diff())      *
data['Volume']).fillna(0).cumsum()
        return data
    data = calculate_obv(data)
    st.subheader("On-Balance Volume (OBV)")
    fig_obv = go.Figure()
    fig_obv.add_trace(go.Scatter(x=data['Date'],      y=data['OBV'],
mode='lines', name='OBV'))
    fig_obv.update_layout(title='On-Balance Volume (OBV)')
    st.plotly_chart(fig_obv, use_container_width=True)
```

DESCRIPTION:

This code segment calculates the On-Balance Volume (OBV) for the given stock data. The `calculate_obv` function takes the stock data as input.

Within the function, it computes the OBV values using the following steps:

1. Calculates the difference in closing prices (`data['Close'].diff()`) and applies the `np.sign` function to get the sign of each difference. This indicates whether the closing price increased (positive value) or decreased (negative value) compared to the previous day.

2. Multiplies the sign of the price difference by the volume traded (`data['Volume']`) for each day to determine whether the volume should be added or subtracted based on the price movement.

3. Fills any NaN values resulting from the first calculation with 0 and then computes the cumulative sum of the OBV values over time using `cumsum`.

After calculating the OBV for the data, it plots the OBV values over time using a Plotly figure (`fig_obv`). The x-axis represents the dates (`data['Date']`), and the y-axis represents the OBV values. The OBV is plotted as a line (`mode='lines'`).

The layout of the figure is updated to set the title to "On-Balance Volume (OBV)". Finally, the OBV chart is rendered in the Streamlit app using `st.plotly_chart(fig_obv, use_container_width=True)`, which displays the chart within the Streamlit interface, utilizing the available container width.

On-Balance Volume (OBV)

INFERENCE:

OBV is a momentum indicator that uses volume flow to predict changes in stock price. It adds volume on up days and subtracts volume on down days. The layout of the figure is updated to set the title to "On-Balance Volume (OBV)". Finally, the OBV chart is rendered in the Streamlit app using st.plotly_chart(fig_obv, use_container_width=True), which displays the chart within the Streamlit interface, utilizing the available container width.

Rising OBV: Indicates buying pressure.

Falling OBV: Indicates selling pressure.

SIMPLE MOVING AVERAGE:

CODE:

```
    def calculate_sma(data, window):

        data[f'SMA_{window}']                                    =
data['Close'].rolling(window=window).mean()

        return data

    data = calculate_sma(data, window=50)

    data = calculate_sma(data, window=200)

    st.subheader("Simple Moving Average (SMA)")

    fig_sma = go.Figure()

    fig_sma.add_trace(go.Scatter(x=data['Date'],     y=data['Close'],
mode='lines', name='Close'))

    fig_sma.add_trace(go.Scatter(x=data['Date'],    y=data['SMA_50'],
mode='lines', name='SMA 50'))

    fig_sma.add_trace(go.Scatter(x=data['Date'],   y=data['SMA_200'],
mode='lines', name='SMA 200'))

    fig_sma.update_layout(title='Simple Moving Average (SMA)')

    st.plotly_chart(fig_sma, use_container_width=True)
```

DESCRIPTION:

This code calculates the Simple Moving Average (SMA) for a given stock dataset. The `calculate_sma` function computes the SMA with a specified window size.

Here's how it works:

1. The `calculate_sma` function takes two parameters: the `data` dataframe and the `window` size for the SMA.

2. Within the function, it computes the SMA for the closing prices (`data['Close']`) using the `rolling` method with the specified window size.

3. The calculated SMA values are stored in new columns named based on the window size (e.g., `SMA_50`, `SMA_200`).

4. After calculating the SMA for both the 50-day and 200-day windows, the function returns the updated dataframe.

The SMA is calculated twice in this case, first with a 50-day window and then with a 200-day window. This is a common practice in technical analysis for comparing short-term and long-term trends.

After calculating the SMAs, the code plots the closing prices along with the SMAs on a Plotly figure (`fig_sma`). It adds three traces to the plot:

- The closing prices as a line (`mode='lines'`, name='Close').

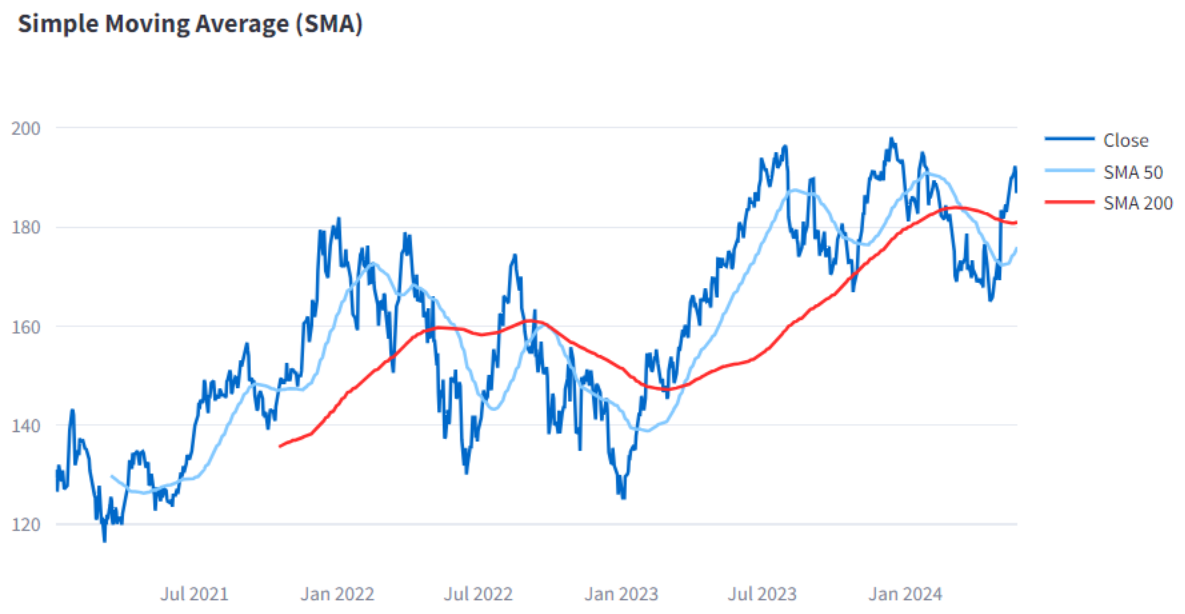
- The 50-day SMA as a line (`mode='lines'`, name='SMA 50').

- The 200-day SMA as a line (`mode='lines'`, name='SMA 200').

The layout of the figure is updated to set the title to "Simple Moving Average (SMA)".

Finally, the SMA chart is rendered in the Streamlit app using `st.plotly_chart(fig_sma, use_container_width=True)`, which displays the chart within the Streamlit interface, utilizing the available container width.

Simple Moving Average (SMA)



INFERENCE:

SMA is the average stock price over a specified number of periods. It smooths out price data to identify trends.

Short-term SMA: More sensitive to recent price changes.

Long-term SMA: More stable and less sensitive to short-term fluctuations.

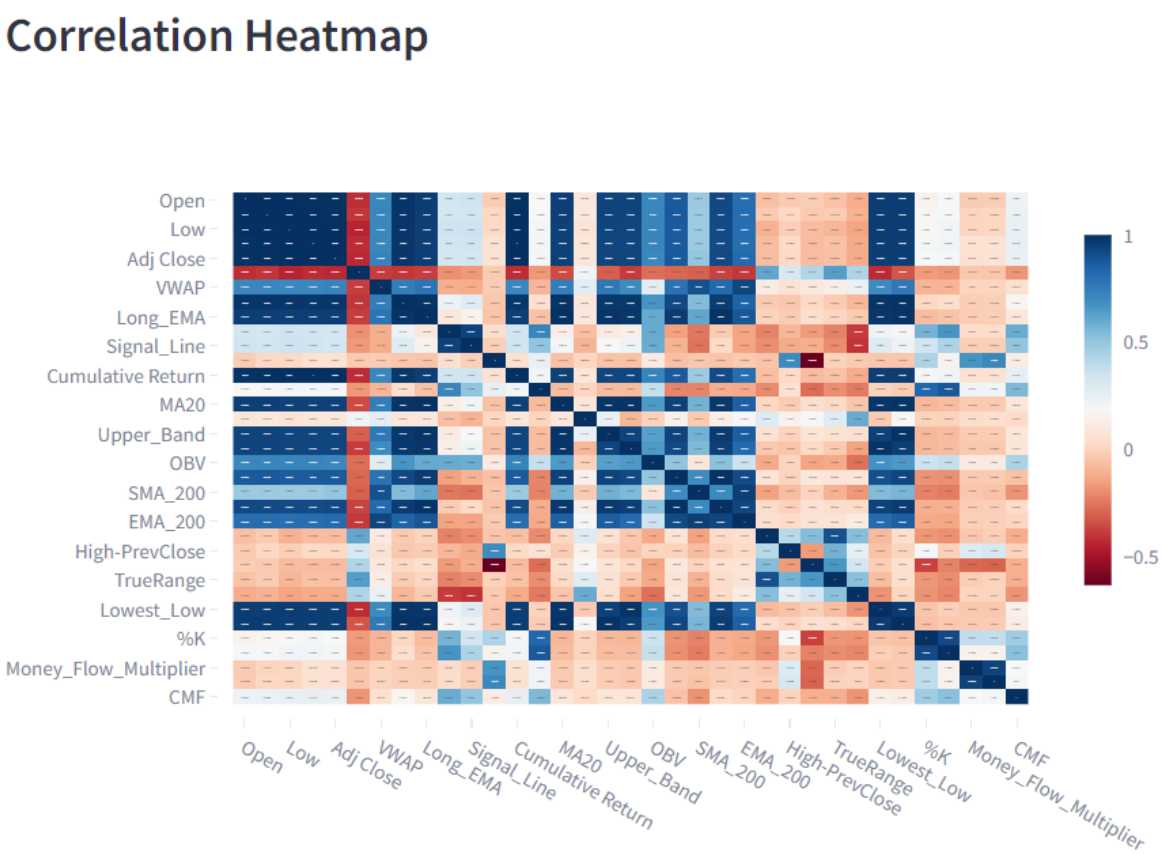
CORRELATION HEATMAP:

CODE:

```
st.subheader("Correlation Heatmap")

corr_matrix = data[numeric_data].corr()

fig_heatmap = px.imshow(corr_matrix, text_auto=True, aspect="auto", color_continuous_scale='RdBu')

st.plotly_chart(fig_heatmap, use_container_width=True)
```

DESCRIPTION:

The code snippet creates and displays a correlation heatmap for numerical data within a Streamlit app. It calculates the correlation matrix for the numerical columns in the dataset, then uses Plotly Express to generate a heatmap with the 'RdBu' color scale. The heatmap includes auto-generated text annotations and is adjusted to fit the container width for better visualization.



INFERENCE:

The correlation heatmap shows the relationships between different numerical features in the dataset. High positive correlations (closer to 1, shown in dark blue) are observed between some moving averages and related indicators, indicating they move together. For example, 'MA20', 'SMA_200', and 'EMA_200' have strong positive correlations with each other. Conversely, high negative correlations (closer to -1, shown in dark red) indicate inverse relationships, such as between 'Signal_Line' and 'CMF'. Features like 'TrueRange' and 'Low' exhibit moderate positive correlations with 'Open' and 'High-PrevClose'. The heatmap reveals these relationships, highlighting which features are closely related and which are inversely related, aiding in understanding the interdependencies within the data.

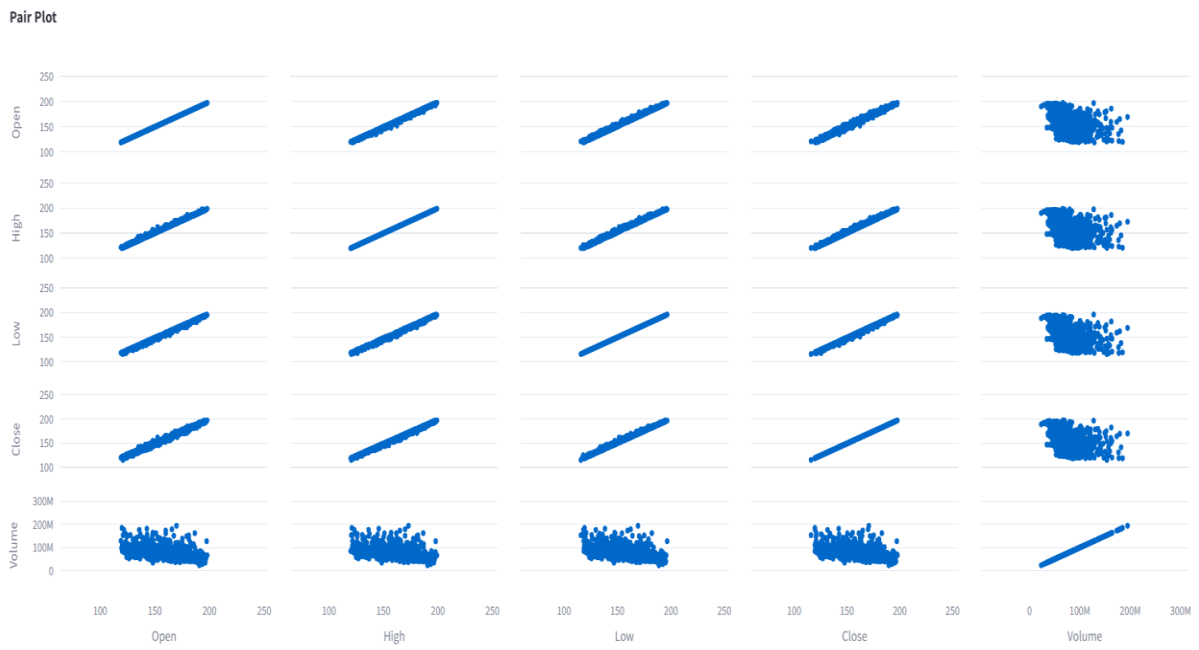
PAIR PLOT:

CODE:

```
st.subheader("Pair Plot")

    # Select fewer important features to avoid congestion

    selected_columns = numeric_d[['Open', 'High', 'Low', 'Close',
'Volume']]

    fig_pairplot = px.scatter_matrix(selected_columns,
dimensions=selected_columns.columns, title="Pair Plot", height=800,
width=800)

    st.plotly_chart(fig_pairplot, use_container_width=True)
```

DESCRIPTION:

The code snippet generates a pair plot to visualize relationships between selected numerical features ('Open', 'High', 'Low', 'Close', 'Volume') in the dataset. It uses Plotly Express to create a scatter matrix, showing pairwise scatter plots and distributions for these features. The plot is configured to have a title "Pair Plot" and dimensions of 800x800 pixels, and it is displayed in the Streamlit app with container width adjustment for optimal viewing.



EXPONENTIAL MOVING AVERAGE(EMA):

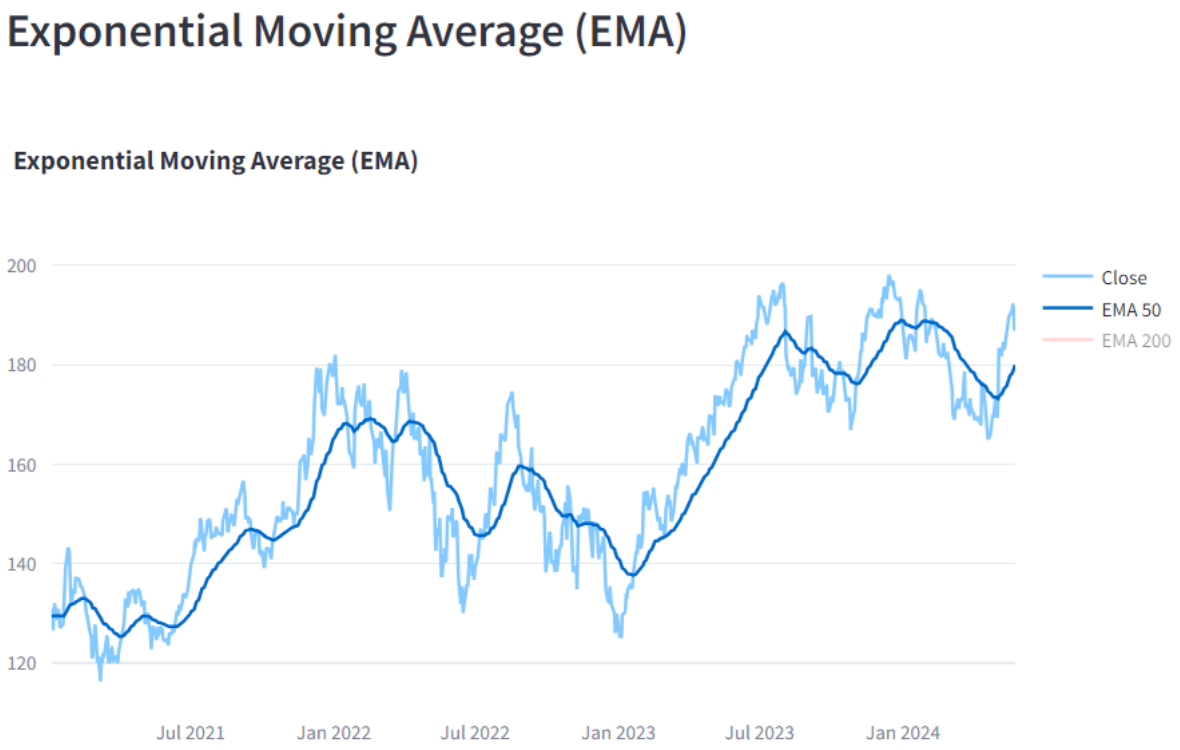
CODE:

```
def calculate_ema(data, window):
    data[f'EMA_{window}'] = data['Close'].ewm(span=window, adjust=False).mean()
    return data
data = calculate_ema(data, window=50)
data = calculate_ema(data, window=200)
st.subheader("Exponential Moving Average (EMA)")
fig_ema = go.Figure()
fig_ema.add_trace(go.Scatter(x=data['Date'], y=data['Close'], mode='lines', name='Close'))
fig_ema.add_trace(go.Scatter(x=data['Date'], y=data['EMA_50'], mode='lines', name='EMA 50'))
```

```
    fig_ema.add_trace(go.Scatter(x=data['Date'],  y=data['EMA_200'],
mode='lines', name='EMA 200'))

    fig_ema.update_layout(title='Exponential Moving Average (EMA)')

    st.plotly_chart(fig_ema,                use_container_width=True)
```

DESCRIPTION:

In this segment, the Exponential Moving Average (EMA) is calculated and visually represented for a selected stock's closing prices within a Streamlit dashboard. Initially, the calculate_ema function is defined to compute the EMA using the provided data and a specified window size. Specifically, EMAs are calculated for both a 50-day and a 200-day window. Subsequently, a subplot titled "Exponential Moving Average (EMA)" is generated using Plotly within the Streamlit dashboard. This subplot comprises three lines: one showcasing the stock's closing prices over time, alongside two others exhibiting the computed EMAs for the 50-day and 200-day windows, respectively. To enhance clarity, the plot is furnished with a descriptive title. Finally, the plot is displayed using st.plotly_chart, ensuring its optimal fit within the Streamlit app's container width. Such visualization offers valuable insights into the underlying trend and smoothing patterns applied to the stock's price data through the EMA computation.



INFERENCE:

Based on the graph, the 50-day and 200-day exponential moving averages (EMAs) are both increasing over time. This suggests that the price of the asset is in an uptrend.

However, it is important to note that EMAs are lagging indicators, meaning that they react to price changes with a delay. So, while the EMAs in the image suggest that the price trend is up, they do not guarantee that the uptrend will continue in the future.

Here are some additional things to consider:

- The slope of the EMAs. A steeper slope suggests a stronger uptrend.
- Volatility. The EMAs will be more volatile if the price of the asset is volatile.
- Other technical indicators. EMAs are often used in conjunction with other technical indicators to confirm signals.

Overall, the exponential moving averages in the image suggest that the price of the asset is in an uptrend. However, it is important to consider other factors before making any investment decisions.

AVERAGE TRUE RANGE:

CODE:

```
def calculate_atr(data, window=14):

        data['High-Low'] = data['High'] - data['Low']

        data['High-PrevClose']      =      np.abs(data['High']      -
data['Close'].shift(1))

        data['Low-PrevClose']      =      np.abs(data['Low']      -
data['Close'].shift(1))

        data['TrueRange'] = data[['High-Low', 'High-PrevClose', 'Low-
PrevClose']].max(axis=1)

        data['ATR']                                                =
data['TrueRange'].rolling(window=window).mean()

        return data

    data = calculate_atr(data)
```

```
    st.subheader("Average True Range (ATR)")

    fig_atr = go.Figure()

    fig_atr.add_trace(go.Scatter(x=data['Date'],      y=data['ATR'],
mode='lines', name='ATR'))

    fig_atr.update_layout(title='Average True Range (ATR)')

    st.plotly_chart(fig_atr, use_container_width=True)
```

DESCRIPTION:

The provided code effectively calculates the Average True Range (ATR), a crucial technical analysis indicator that quantifies market volatility. By decomposing the range of price movement, the ATR provides traders and analysts with insights into price fluctuations over a specified period. The core of the calculation lies in the `calculate_atr` function. This function processes a DataFrame containing stock price data, specifically columns for 'High', 'Low', and 'Close' prices. It computes the True Range (TR), which is the greatest of the following: the current high minus the current low, the absolute value of the current high minus the previous close, and the absolute value of the current low minus the previous close. This computation is repeated for each period in the dataset, and the rolling mean of these True Range values over a defined window (defaulting to 14 periods) yields the ATR.

Upon calculating the ATR, the code proceeds to visualize it using Plotly within a Streamlit application. A subheader titled "Average True Range (ATR)" is introduced to provide context. The visualization is achieved by creating a scatter plot where the x-axis represents the date and the y-axis denotes the ATR values. This graphical representation is dynamically rendered in the Streamlit app using the `st.plotly_chart` function, which ensures an interactive and responsive display.

This implementation not only demonstrates the ATR's utility in assessing market volatility but also showcases how modern tools like Plotly and Streamlit can be harnessed to create insightful and user-friendly analytical dashboards. The ability to visualize ATR trends over time enables traders to identify periods of high and low volatility, aiding in the development of more informed trading strategies.

Average True Range (ATR)

INFERENCE:

The Average True Range (ATR) chart indicates fluctuating market volatility from mid-2021 to early 2024, with periods of high volatility peaking around mid-2021 and early 2022, followed by reduced volatility mid-2022 to early 2023, and a resurgence in mid-2023. The recent trend towards early 2024 shows increasing volatility. These cyclical patterns suggest traders should be prepared for varying market conditions, with higher ATR values implying greater price movements and risk, necessitating cautious and adaptable trading strategies.

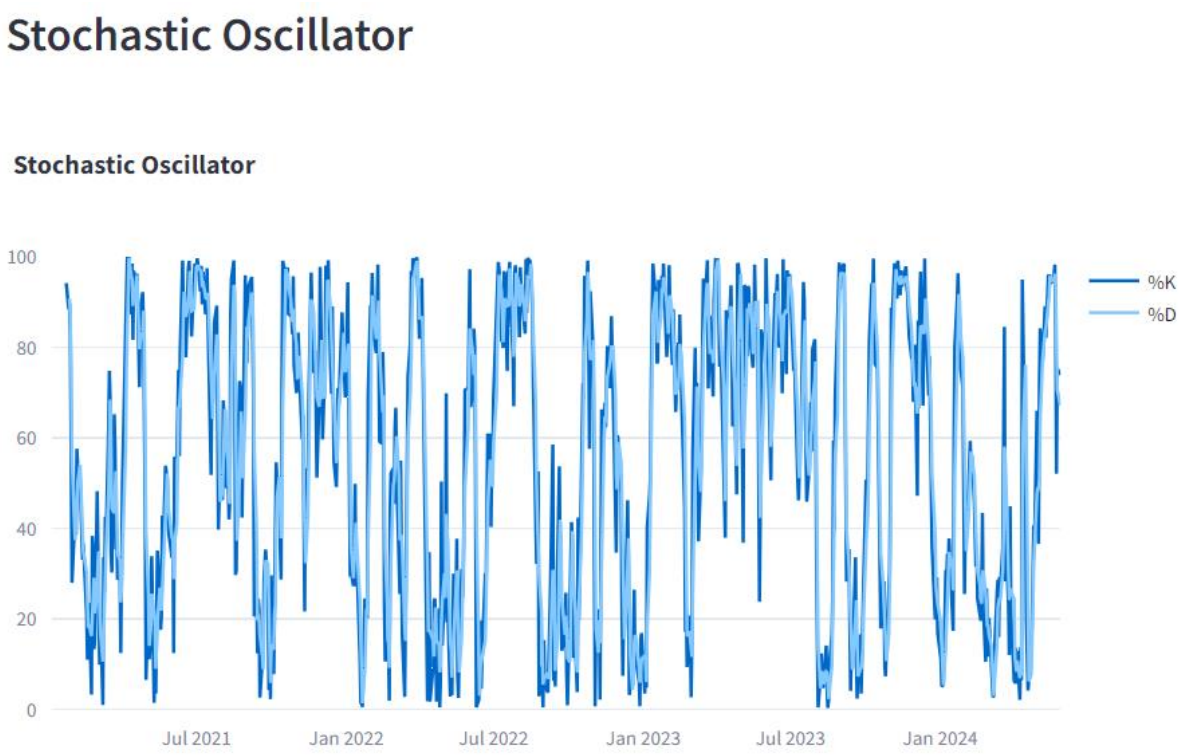
STOCHASTIC OSCILLATOR:

CODE:

```
    def calculate_stochastic_oscillator(data, window=14):

        data['Lowest_Low']                                          =
data['Low'].rolling(window=window).min()

        data['Highest_High']                                        =
data['High'].rolling(window=window).max()

        data['%K'] = 100 * ((data['Close'] - data['Lowest_Low']) /
(data['Highest_High'] - data['Lowest_Low']))

        data['%D'] = data['%K'].rolling(window=3).mean()

        return data


    data = calculate_stochastic_oscillator(data)


    st.subheader("Stochastic Oscillator")

    fig_stochastic = go.Figure()

    fig_stochastic.add_trace(go.Scatter(x=data['Date'],
y=data['%K'], mode='lines', name='%K'))

    fig_stochastic.add_trace(go.Scatter(x=data['Date'],
y=data['%D'], mode='lines', name='%D'))

    fig_stochastic.update_layout(title='Stochastic      Oscillator',
yaxis=dict(range=[0, 100]))

    st.plotly_chart(fig_stochastic, use_container_width=True)
```

DESCRIPTION:

Focusing on the Stochastic Oscillator computation and visualization within a Streamlit dashboard, this code segment commences with the definition of the calculate_stochastic_oscillator function. This function is pivotal for deriving the Stochastic Oscillator values based on the provided stock data. Utilizing rolling windows, it computes the lowest low and highest high prices over a specified period, typically set to 14 days. Subsequently, employing the formula %K = 100 * ((Close - Lowest_Low) / (Highest_High - Lowest_Low)), it calculates the %K value. Additionally, %D value is determined by taking the rolling mean of %K over a window of 3 periods. Once these Stochastic Oscillator values are computed, the code visualizes them within the Streamlit dashboard using Plotly. The resulting plot, labeled "Stochastic Oscillator," portrays both %K and %D values plotted against corresponding dates. For clarity, the y-axis range is constrained between 0 and 100. This visualization proves invaluable for evaluating momentum and identifying potential trend reversals in the selected stock's price movements over time.



INFERENCE:

The Stochastic Oscillator chart from mid-2021 to early 2024 shows frequent oscillations of the %K and %D lines between 0 and 100, indicating rapid changes in market momentum. The asset frequently enters overbought (>80) and oversold (<20) conditions, suggesting multiple potential buying and selling opportunities. The frequent crossings of %K and %D lines reflect a highly volatile market with numerous short-term fluctuations, providing traders with many signals for potential entries and exits based on momentum shifts.

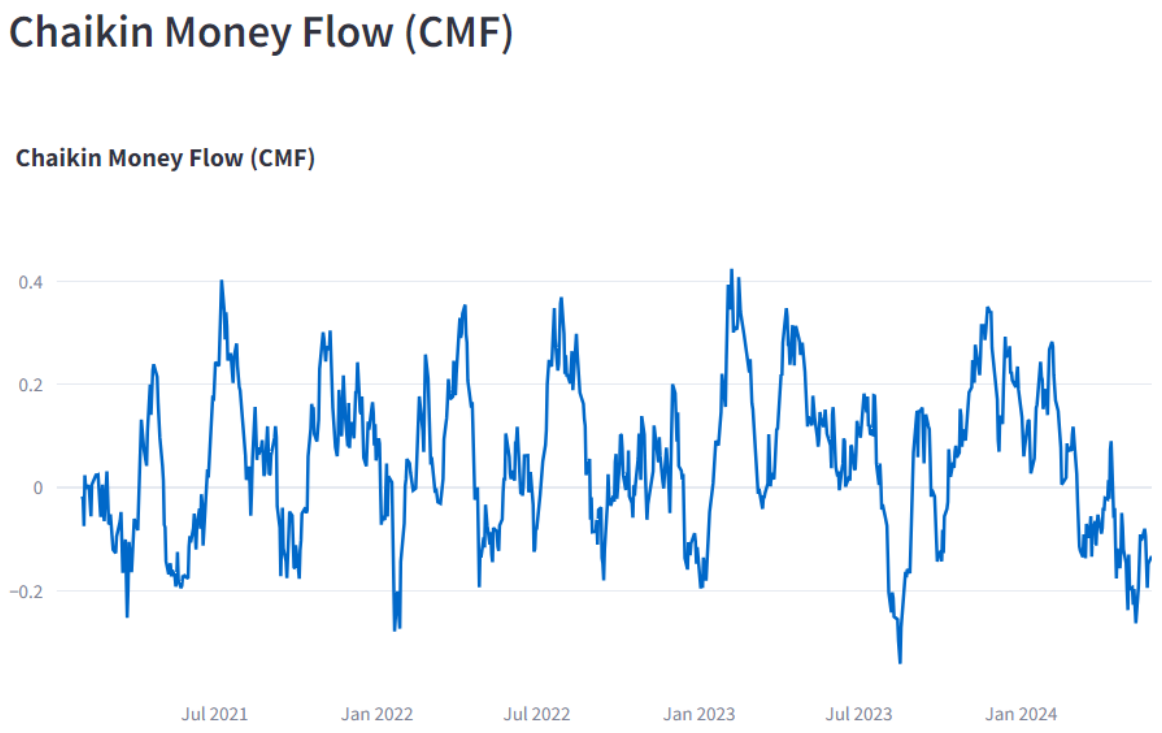
CHAIKIN MONEY FLOW:

CODE:

```
def calculate_cmf(data, window=20):
        data['Money_Flow_Multiplier'] = ((data['Close'] -
data['Low']) - (data['High'] - data['Close'])) / (data['High'] -
data['Low'])
        data['Money_Flow_Volume'] = data['Money_Flow_Multiplier'] *
data['Volume']
        data['CMF'] =
data['Money_Flow_Volume'].rolling(window=window).sum() /
data['Volume'].rolling(window=window).sum()
        return data
    data = calculate_cmf(data)
    st.subheader("Chaikin Money Flow (CMF)")
    fig_cmf = go.Figure()
    fig_cmf.add_trace(go.Scatter(x=data['Date'], y=data['CMF'],
mode='lines', name='CMF'))
    fig_cmf.update_layout(title='Chaikin Money Flow (CMF)')
    st.plotly_chart(fig_cmf, use_container_width=True)
```

DESCRIPTION:

Focusing on the Chaikin Money Flow (CMF) computation and visualization within a Streamlit dashboard, this code segment initiates with the definition of the calculate_cmf function. This function is pivotal for deriving the CMF values based on the provided stock data. It computes the Money Flow Multiplier, which represents the proportion of the day's trading activity occurring at the top or bottom of the day's price range. Subsequently, it calculates the Money Flow Volume, which is the Money Flow Multiplier multiplied by the day's volume. Utilizing rolling windows, the CMF is then determined by summing the Money Flow Volume over a specified period and dividing it by the sum of volumes over the same period. Once these CMF values are computed, the code visualizes them within the Streamlit dashboard using Plotly. The resulting plot, labeled "Chaikin Money Flow (CMF)," depicts the CMF values plotted against corresponding dates. For clarity, the plot's title is set to "Chaikin Money Flow (CMF)." This visualization is instrumental in assessing the flow of money into or out of the selected stock, aiding in identifying potential trends and market sentiment shifts over time.



INFERENCE:

The Chaikin Money Flow (CMF) chart from mid-2021 to early 2024 indicates fluctuating levels of buying and selling pressure, with values oscillating between 0.4 and -0.2. Positive CMF values suggest periods of accumulation (buying pressure), while negative values indicate distribution (selling pressure). The chart shows several peaks and troughs, reflecting shifts in market sentiment. Notably, there are more frequent and deeper dips into negative territory in recent months, suggesting increasing selling pressure and potential bearish sentiment in early 2024. Overall, the CMF highlights significant volatility and changes in market dynamics over the observed period.

BINNING:

CODE:

```
    bin_feature = st.sidebar.selectbox("Select feature for
binning:", data.columns)

    num_bins = st.sidebar.slider("Number of bins:", min_value=2,
max_value=20, value=10)

    binned_data, bin_edges =
np.histogram(data[bin_feature].astype(float), bins=num_bins)
```

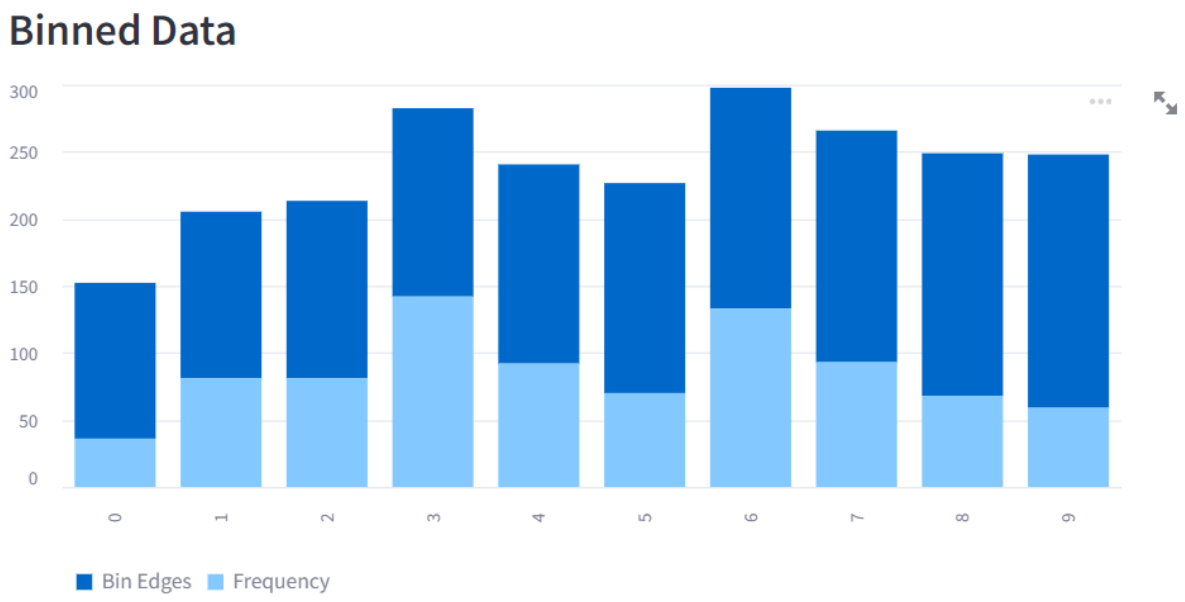
DESCRIPTION:

The given snippet integrates interactive elements into a Streamlit application, allowing users to perform binning on a selected feature from a dataset. Through Streamlit's sidebar widgets, users can dynamically choose a feature for binning and specify the number of bins to segment the data.

First, the `st.sidebar.selectbox` function creates a dropdown menu in the sidebar, populated with the column names of the `data` DataFrame. This dropdown allows users to select which feature (column) they want to bin. Next, a slider is implemented using `st.sidebar.slider`, enabling users to set the desired number of bins for the histogram, with a range from 2 to 20 and a default value of 10.

Once the user selects the feature and the number of bins, the code employs NumPy's `np.histogram` function to perform the binning. This function takes the selected feature (converted to float) and the specified number of bins, returning two outputs: `binned_data`, which contains the counts of data points in each bin, and `bin_edges`, which represents the edges of the bins.

This interactive approach allows users to customize their analysis directly within the Streamlit app, facilitating an intuitive exploration of data distributions and enabling more tailored insights.



VOLUME TRADED OVER TIME:

CODE:

```
st.subheader("Closing Price Over Time (with Animation)")

    fig_close_animated = go.Figure()

    fig_close_animated.add_trace(go.Scatter(x=data['Date'],
y=data['Close'], mode='lines', name='Close'))

    fig_close_animated.update_layout(title='Closing Price Over Time
(with Animation)')

    fig_close_animated.update_traces(

        line=dict(color='blue', width=1),

        selector=dict(mode='lines')

    )

    fig_close_animated.update_layout(

        xaxis=dict(range=[data['Date'].min(), data['Date'].max()]),

        yaxis=dict(range=[data['Close'].min(),
data['Close'].max()]),

        updatemenus=[

            {

                "buttons": [

                    {

                        "args": [None, {"frame": {"duration": 500,
"redraw": True}, "fromcurrent": True}],

                        "label": "Play",

                        "method": "animate"

                    },
```

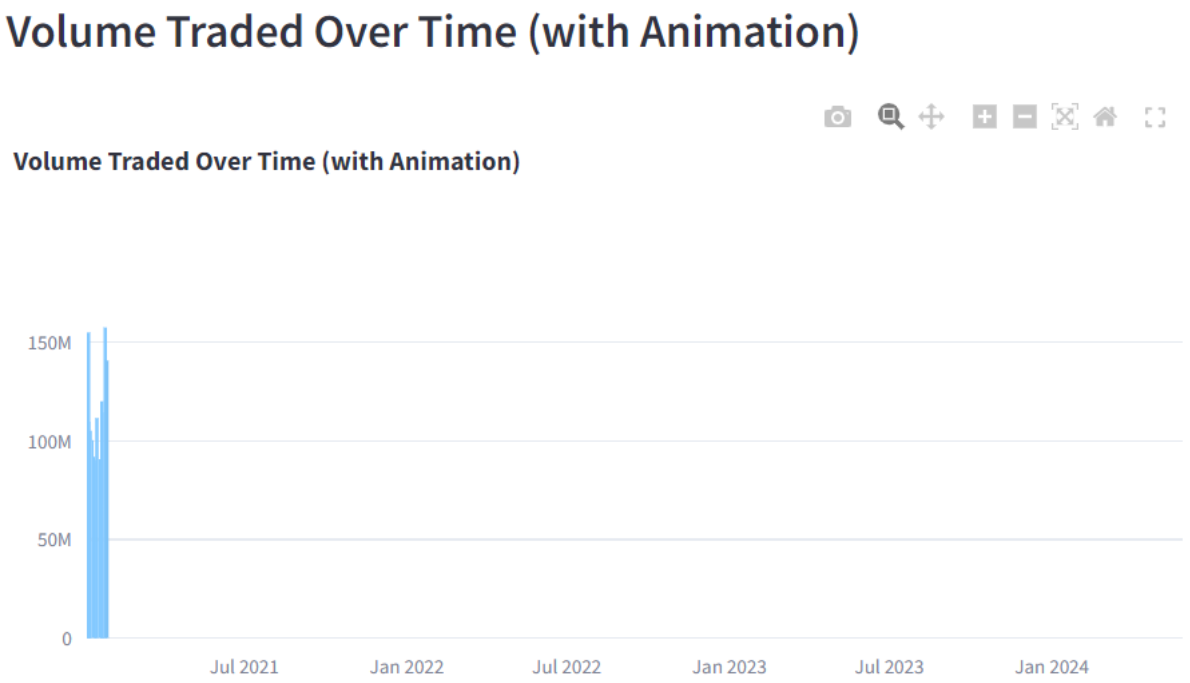
```
                    {
                        "args": [[None], {"frame": {"duration": 0,
"redraw": True}, "mode": "immediate", "transition": {"duration":
0}}],
                        "label": "Pause",
                        "method": "animate"
                    }
                ],
                "direction": "left",
                "pad": {"r": 10, "t": 87},
                "showactive": False,
                "type": "buttons",
                "x": 0.1,
                "xanchor": "right",
                "y": 0,
                "yanchor": "top"
            }
        ]
    )

    frames = [go.Frame(data=[go.Scatter(x=data['Date'][:k+1],
y=data['Close'][:k+1])], name=str(k)) for k in range(1,
len(data['Date']))]
    fig_close_animated.frames = frames
```

```
    st.plotly_chart(fig_close_animated, use_container_width=True)


    st.subheader("Volume Traded Over Time (with Animation)")
    fig_volume_animated = go.Figure()
    fig_volume_animated.add_trace(go.Bar(x=data['Date'],
y=data['Volume'], name='Volume'))
    fig_volume_animated.update_layout(title='Volume Traded Over Time
(with Animation)')
    fig_volume_animated.update_traces(
        selector=dict(type='bar')
    )
    fig_volume_animated.update_layout(
        xaxis=dict(range=[data['Date'].min(), data['Date'].max()]),
        yaxis=dict(range=[0, data['Volume'].max()]),
        updatemenus=[
            {
                "buttons": [
                    {
                        "args": [None, {"frame": {"duration": 500,
"redraw": True}, "fromcurrent": True}],
                        "label": "Play",
                        "method": "animate"
                    },
                    {
```

```
                        "args": [[None], {"frame": {"duration": 0,
"redraw": True}, "mode": "immediate", "transition": {"duration":
0}}],
                        "label": "Pause",
                        "method": "animate"
                    }
                ],
                "direction": "left",
                "pad": {"r": 10, "t": 87},
                "showactive": False,
                "type": "buttons",
                "x": 0.1,
                "xanchor": "right",
                "y": 0,
                "yanchor": "top"
            }
        ]
    )

    frames = [go.Frame(data=[go.Bar(x=data['Date'][:k+1],
y=data['Volume'][:k+1])], name=str(k)) for k in range(1,
len(data['Date']))]
    fig_volume_animated.frames = frames

    st.plotly_chart(fig_volume_animated, use_container_width=True)
```

DESCRIPTION:

The provided code creates two animated plots using Plotly within a Streamlit application. The first plot, titled "Closing Price Over Time (with Animation)," uses a line chart to display the closing prices of a dataset over time, with play and pause buttons to control the animation. The second plot, "Volume Traded Over Time (with Animation)," employs a bar chart to illustrate the trading volume over time, also featuring animation controls. Both plots are configured to update dynamically, revealing data points incrementally to visually represent changes over the specified time periods. The animation frames for each plot are generated by iterating through the dataset, adding new data points in each frame to create a smooth transition effect.



INFERENCE:

The "Volume Traded Over Time (with Animation)" plot indicates a significant trading volume around mid-2021, which rapidly decreases, resulting in minimal trading activity from late 2021 through early 2024. This suggests that there was a peak in trading activity in mid-2021, followed by a dramatic decline, leading to consistently low trading volumes in the subsequent years. This pattern may reflect market events, changes in investor behavior, or other factors affecting the trading volume over this period.

3D SCATTER PLOT OF HIGH, LOW AND CLOSE PRICES:

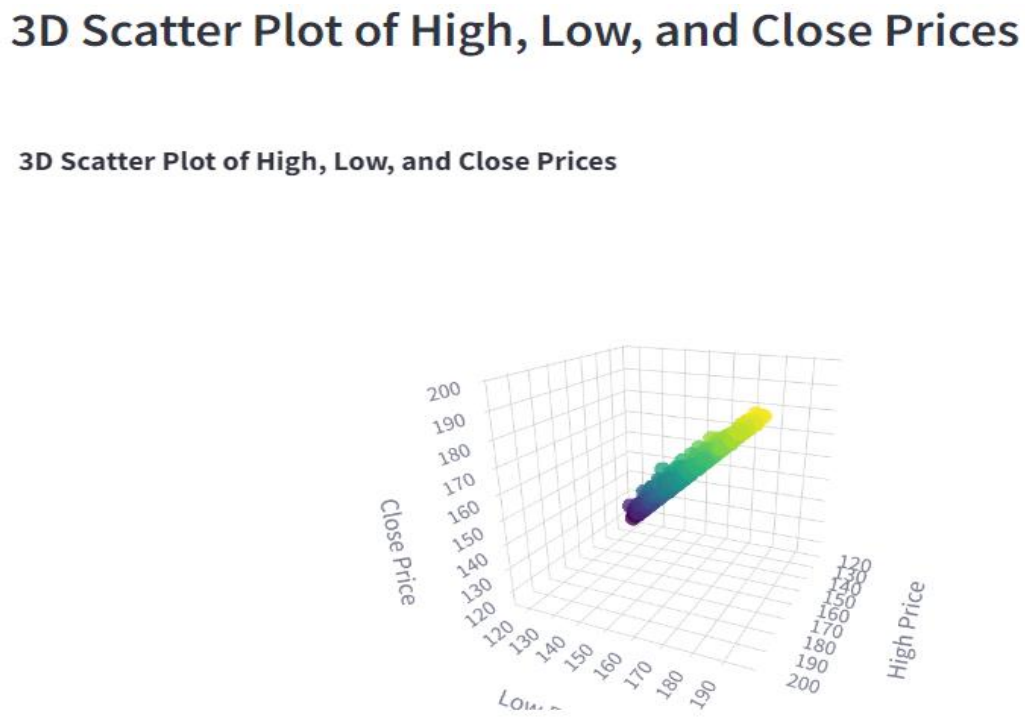
CODE:

```
# 3-D Interactive plot for high, low and close prices

    st.subheader("3D Scatter Plot of High, Low, and Close Prices")

    fig_3d_scatter = go.Figure(

        data=[go.Scatter3d(

            x=data['High'],

            y=data['Low'],

            z=data['Close'],

            mode='markers',

            marker=dict(

                size=5,

                color=data['Close'],                # set color to an
array/list of desired values

                colorscale='Viridis',                     # choose a
colorscale

                opacity=0.8

            )

        )]

    )

    fig_3d_scatter.update_layout(

        title="3D Scatter Plot of High, Low, and Close Prices",

        scene=dict(

            xaxis_title='High Price',
```

```
            yaxis_title='Low Price',
            zaxis_title='Close Price'
        )
    )
    st.plotly_chart(fig_3d_scatter, use_container_width=True)


    st.subheader("3D Surface Plot of Close Prices Over Time")
```

DESCRIPTION:

The code snippet generates a 3D scatter plot using Plotly in a Streamlit application to visualize the relationship between high, low, and close prices of financial data. The scatter plot uses `data['High']`, `data['Low']`, and `data['Close']` for the x, y, and z axes, respectively, with marker colors representing close prices on a 'Viridis' colorscale. The plot includes axis titles and is displayed within the Streamlit app using `st.plotly_chart`. Additionally, a subheader is set up for an upcoming 3D surface plot of close prices over time, although its implementation is not included in the snippet.



INFERENCE:

These plots offer a comprehensive visualization of financial data: the 3D scatter plot illustrates the relationship between high, low, and close prices, allowing for the identification of patterns and clusters indicative of market behavior, while the 3D surface plot charts the trend of close prices over time, aiding in the assessment of overall market movement. Together, they empower analysts and traders with valuable insights for making informed decisions in financial markets.

NORMALIZATION:

CODE:

```
    normalize_data = st.sidebar.checkbox("Normalize data")

    if normalize_data:

        scaler = MinMaxScaler()

        data_normalized = pd.DataFrame(scaler.fit_transform(data),
columns=data.columns)

    else:

        data_normalized = data
```

DESCRIPTION:

The given snippet integrates interactive elements into a Streamlit application, allowing users to perform binning on a selected feature from a dataset. Through Streamlit's sidebar widgets, users can dynamically choose a feature for binning and specify the number of bins to segment the data.

First, the `st.sidebar.selectbox` function creates a dropdown menu in the sidebar, populated with the column names of the `data` DataFrame. This dropdown allows users to select which feature (column) they want to bin. Next, a slider is implemented using `st.sidebar.slider`, enabling users to set the desired number of bins for the histogram, with a range from 2 to 20 and a default value of 10.

Once the user selects the feature and the number of bins, the code employs NumPy's `np.histogram` function to perform the binning. This function takes the selected feature (converted to float) and the specified number of bins, returning two outputs: `binned_data`, which contains the counts of data points in each bin, and `bin_edges`, which represents the edges of the bins.

This interactive approach allows users to customize their analysis directly within the Streamlit app, facilitating an intuitive exploration of data distributions and enabling more tailored insights.

Normalized Data

	Open	High	Low	Close	Adj Close	Volume	VWAP	Short_EMA	Long_EMA	MACD	Sign
0	0.1834	0.1668	0.1306	0.1596	0.1514	0.6958	0.0752	0.1078	0.0908	0.5032	
1	0.1248	0.1431	0.1513	0.1792	0.1702	0.4295	0.082	0.1112	0.0925	0.5128	
2	0.11	0.1344	0.1259	0.1253	0.1183	0.7646	0.0524	0.1048	0.0894	0.4937	
3	0.1181	0.1418	0.1442	0.1781	0.1691	0.4991	0.059	0.1084	0.0911	0.5048	
4	0.1696	0.1544	0.1735	0.1919	0.1824	0.4733	0.0723	0.1138	0.0939	0.5203	
5	0.1286	0.1233	0.1521	0.1544	0.1463	0.4454	0.0692	0.112	0.0932	0.5138	
6	0.1199	0.1173	0.1318	0.1522	0.1442	0.3962	0.0638	0.1101	0.0924	0.5075	
7	0.1232	0.1395	0.152	0.1777	0.1687	0.3769	0.0662	0.1128	0.0938	0.5151	
8	0.149	0.1338	0.1553	0.1535	0.1455	0.3861	0.0658	0.111	0.0931	0.509	
9	0.1234	0.124	0.1336	0.1319	0.1247	0.5108	0.0601	0.1058	0.0905	0.4935	

SAMPLING:

CODE:

```
    sampling_technique = st.sidebar.selectbox("Sampling technique:",
["None", "Random Sampling", "Stratified Sampling"])

    if sampling_technique == "Random Sampling":
```

```
        sample_size = st.sidebar.slider("Sample size:", min_value=1, max_value=len(data), value=len(data)//2)
        sampled_data = data.sample(sample_size)
    elif sampling_technique == "Stratified Sampling":
        stratify_by = st.sidebar.selectbox("Select feature for stratification:", data.columns)
        if stratify_by not in data.columns:
            st.error(f"Selected column '{stratify_by}' does not exist in the dataset.")
            sampled_data = data
        else:
            if sample_size > len(data):
                st.error("Sample size cannot be larger than the dataset size.")
                sample_size = len(data)
            sampled_data = data.groupby(stratify_by, group_keys=False).apply(lambda x: x.sample(min(len(x), sample_size)))
    else:
        sampled_data = data
```

DESCRIPTION:

The provided code snippet incorporates interactive elements in a Streamlit application, enabling users to select and apply different sampling techniques on a dataset. Using Streamlit's sidebar widgets, users can choose between "None", "Random Sampling", and "Stratified Sampling" to sample the data, adjusting parameters as needed.

The `st.sidebar.selectbox` function creates a dropdown menu in the sidebar, offering a selection of sampling techniques. If the user chooses "Random Sampling", a slider appears via `st.sidebar.slider`, allowing the user to specify the sample size. The `data.sample` method then randomly selects the specified number of rows from the dataset.

For "Stratified Sampling", another dropdown menu allows the user to choose a feature for stratification from the dataset's columns. If the selected column doesn't exist, an error message is displayed using `st.error`, and the original dataset is used. If the column is valid and the sample size does not exceed the dataset size, the dataset is grouped by the chosen feature, and the `sample` method is applied within each group to maintain proportional representation.

If "None" is selected, no sampling is applied, and the entire dataset is used as is. This interactive setup allows users to dynamically apply sampling techniques within the Streamlit app, facilitating flexible and tailored data analysis.

Sampled Data

	Date	Open	High	Low	Close	Adj Close	Volume	VWAP	Short_EMA
757	2024-01-08 00:00:00	182.09	185.6	181.5	185.56	185.0725	59,144,500	153.4482	188.0964
77	2021-04-26 00:00:00	134.83	135.06	133.56	134.72	132.2315	66,905,100	128.5436	132.4863
26	2021-02-10 00:00:00	136.48	136.99	134.4	135.39	132.8891	73,046,600	133.8879	135.8092
715	2023-11-06 00:00:00	176.38	179.43	176.21	179.23	178.5239	63,841,300	152.0327	174.5729
650	2023-08-04 00:00:00	185.52	187.38	181.92	181.99	181.0286	115,799,700	150.2732	191.5887
342	2022-05-12 00:00:00	142.77	146.2	138.8	142.56	140.9652	182,602,000	147.5958	154.6469
340	2022-05-10 00:00:00	155.52	156.74	152.93	154.51	152.7815	115,366,700	147.6179	158.7253
831	2024-04-24 00:00:00	166.54	169.3	166.21	169.02	168.7911	48,251,800	155.1622	168.6768
517	2023-01-24 00:00:00	140.31	143.16	140.3	142.53	141.3654	66,435,100	147.1317	136.1346
210	2021-11-02 00:00:00	148.66	151.57	148.65	150.02	147.72	69,122,000	135.258	148.7105

CONCLUSION:

In conclusion, "Quantum Quotient Analytics" presents a comprehensive and versatile platform for analyzing stock market data. Through a meticulous methodology encompassing data acquisition, processing, visualization, and analysis, this tool offers users a wealth of insights into market trends, technical indicators, and predictive modeling.

With a user-friendly interface and interactive visualizations, users can explore stock data dynamically, gaining a deeper understanding of market behavior and potential trading opportunities. The incorporation of technical analysis techniques, such as Bollinger Bands, MACD, and RSI, further enhances the tool's utility for making informed investment decisions.

Moreover, the integration of machine learning modeling enables users to predict closing prices based on selected features, providing valuable insights into future market trends. By evaluating model accuracy metrics, users can assess the reliability of predictions and refine their investment strategies accordingly.

Overall, "Quantum Quotient Analytics" serves as a powerful ally for financial analysts, traders, and enthusiasts, empowering them with the tools and insights needed to navigate the complexities of the stock market and make informed decisions with confidence.